

Trabalho de Laboratório — Padrão Observer em Java

Objetivo

Implementar, em Java, um pequeno sistema de **notificação de um jornal digital (tipo Le Figaro)**, baseado no padrão de projeto **Observer**, seguindo o modelo apresentado em aula.

O objetivo é mostrar que um objeto pode **notificar automaticamente vários outros** quando seu estado muda, sem conhecer suas implementações concretas.

Contexto

Considere um sistema de **jornal digital acessado por celular**, no qual:

- Um jornal publica notícias em tempo real
- Usuários recebem notificações no aplicativo

O sistema deve permitir:

- inscrição de usuários
- cancelamento de inscrição
- notificação automática quando há nova notícia

Diagrama de referência

O sistema deve seguir a estrutura discutida em aula, com:

Interface principal

- JornalDigital (Subject)
- AssinanteApp (Observer)

Classes concretas

- LeFigaroDigital (Subject concreto)

Observadores concretos

- AppCelular
- AppTablet
- AppWeb

Parte 1 — Implementação das interfaces

Implemente as seguintes interfaces:

```
interface JornalDigital
```

```
interface AssinanteApp
```

Métodos esperados

JornalDigital

- inscrever(...)
- cancelar(...)
- notificar()

AssinanteApp

- atualizar(String noticia)

Parte 2 — Implementação do Subject concreto

Implemente a classe: LeFigaroDigital

Atributos

- lista de assinantes
- uma notícia atual

Métodos

- inscrever(...)
- cancelar(...)
- notificar()
- novaNoticia(String noticia)

 Regra importante:

Sempre que uma nova notícia for publicada, o método notificar() deve ser chamado automaticamente.

Parte 3 — Implementação dos Observers

Implemente as classes:

- AppCelular
- AppTablet
- AppWeb

Todas devem implementar a interface AssinanteApp.

Comportamento esperado

Cada classe deve exibir uma mensagem com System.out.println() ao receber uma notícia.

Exemplos:

- "🔔 Usuário João recebeu no celular: Nova notícia"
- "📱 Tablet de Maria atualizou: Nova notícia"
- "🌐 Web atualizou: Nova notícia"

Parte 4 — Funcionamento do sistema

O sistema deve permitir:

- adicionar assinantes
- remover assinantes
- notificar todos automaticamente

Parte 5 — Classe de teste 1

Crie uma classe de teste contendo:

- um objeto LeFigaroDigital
- vários assinantes (de tipos diferentes)

Operações esperadas

1. Registrar todos os assinantes
2. Publicar uma notícia
3. Mostrar que todos recebem

Parte 6 — Classe de teste 2

Criar uma segunda classe de teste para demonstrar:

- cancelamento de assinatura

Exemplo de ideia

1. Registrar 2 ou 3 usuários
2. Publicar uma notícia
3. Remover um usuário
4. Publicar outra notícia

Objetivo:

Mostrar que apenas os usuários restantes recebem a notificação

O que deve ser entregue

O aluno deverá entregar o Diagrama de classes em UML.

Restrição importante

- ! Não incluir classes de teste no diagrama.

Elementos obrigatórios do diagrama

Seu diagrama deve conter **todas as classes/interfaces abaixo**, com seus respectivos métodos e atributos.

1. Interface

JornalDigital

(Subject)

Deve conter os seguintes métodos públicos:

- inscrever(a: AssinanteApp): void
- cancelar(a: AssinanteApp): void
- notificar(): void

2. Interface

AssinanteApp

(Observer)

Deve conter o seguinte método público:

- atualizar(noticia: String): void

. Classe

LeFigaroDigital

(Subject concreto)

Deve:

- implementar JornalDigital

Atributos:

- assinantes: List<AssinanteApp>
- noticia: String

Métodos:

- inscrever(...)
- cancelar(...)
- notificar()
- novaNoticia(noticia: String): void

4. Classes observadoras concretas

Devem implementar AssinanteApp:

- AppCelular
- AppTablet
- AppWeb

Cada uma deve possuir:

Atributo:

- usuario: String

Métodos:

- construtor com parâmetro usuario
- atualizar(noticia: String): void



Relações que devem aparecer no diagrama

O diagrama deve representar explicitamente:

✓ Implementação de interfaces

- LeFigaroDigital → implementa JornalDigital
- AppCelular, AppTablet, AppWeb → implementam AssinanteApp

✓ Associação principal (essencial do padrão)

- LeFigaroDigital mantém uma coleção de AssinanteApp

👉 Representar como:

1 ----- 0..*

LeFigaroDigital AssinanteApp

✓ Indicar multiplicidade corretamente

✓ Indicar que é uma coleção (lista)

✓ Dependência funcional

- LeFigaroDigital chama atualizar() nos observadores

(esta relação pode ser implícita via associação, mas pode ser indicada se desejar maior precisão)

? Perguntas de reflexão

1. Onde está a relação 1 → N no sistema?
2. Onde está a interface sendo utilizada?
3. Onde ocorre o desacoplamento?
4. Qual o papel do método notificar()?
5. Por que esse projeto é um exemplo do padrão Observer?

! Restrições

- Não usar if ou switch para decidir quem recebe notificações
- O sujeito não deve conhecer classes concretas dos observadores
- O sistema deve funcionar apenas via interfaces
- O código deve compilar e executar corretamente
- Implementação obrigatoriamente em Java